

Final Project AISE

Célestin HANS

Due date: *February 01, 2026*

1 - Visualizing Loss Landscapes: PINNs vs. Data-Driven

Task 1 - Data Generation

We generate a dataset of input-output pairs $(f^{(i)}, u^{(i)})$ on the domain $D = [0, 1]^2$ discretized on an $N \times N$ grid (we use $N = 64$). Each sample is created by drawing random coefficients $a_{ij} \sim \mathcal{N}(0, 1)$ and constructing a source field $f(x, y)$ as a truncated Fourier series with complexity level $K \in \{1, 4, 8, 16\}$. The corresponding solution field $u(x, y)$ is the solution of the Poisson problem

$$-\Delta u(x, y) = f(x, y), \quad u|_{\partial D} = 0.$$

We report three examples per K in the dataset (in the results/part1/figures folder), Figure 1 shows one representative example for each value of K .

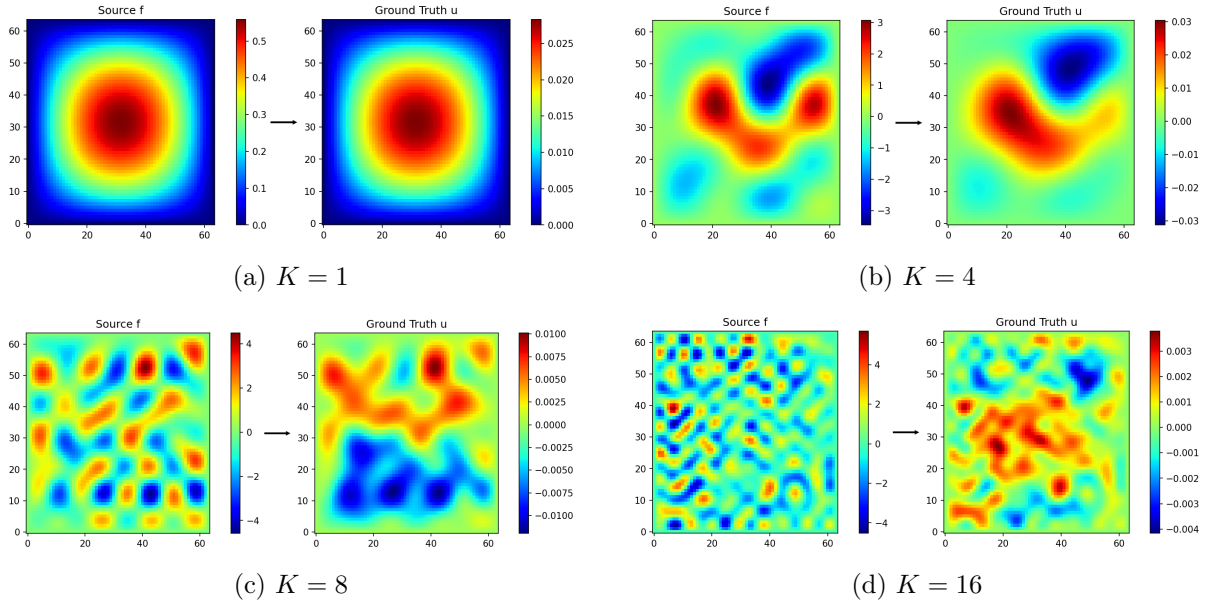


Figure 1: Representative samples for each complexity level $K \in \{1, 4, 8, 16\}$. Each panel shows the input source field f and the corresponding ground-truth solution u .

Physical interpretation. The source field f is a superposition of sinusoidal modes: increasing K injects higher spatial frequencies, so the patterns become more fragmented. We can see this as a growing number of localized “hot/cold” regions and sharper transitions when moving from $K = 1$ to $K = 16$. The solution u is obtained by inverting the Laplacian, this acts as a smoothing (low-pass) operator, attenuating high-frequency components. Therefore, u is smoother than f (except for $K = 1$ where u is proportional to f) and its amplitude is smaller.

Task 2 - Implementation and Training (Data-Driven vs. PINN)

Training procedure. For each complexity level $K \in \{1, 4, 16\}$, we select the representative sample `example_0` generated in Task 1.1 and train a standard Multi-Layer Perceptron (MLP) $u_\theta : \mathbb{R}^2 \rightarrow \mathbb{R}$ to approximate the solution field $u(x, y)$ from spatial coordinates (x, y) . We train two models per K : a **Data-Driven** model minimizing $\mathcal{L}_{\text{Data}}(\theta)$ and a **PINN** model minimizing $\mathcal{L}_{\text{PINN}}(\theta)$.

We use `tanh` as the activation function because PINN training requires stable higher-order derivatives: the PDE residual involves the Laplacian $\Delta \hat{u}_\theta$, hence second derivatives with respect to (x, y) . Smooth activations like `tanh` provide well-behaved derivatives, whereas piecewise-linear activations (e.g., ReLU) can lead to poor or unstable second-derivative signals. For a fair comparison, we keep the same activation for the Data-Driven models.

We first optimize with Adam for fast initial convergence, then switch to L-BFGS for fine-tuning. During Adam training, we apply an early-stopping criterion based on a patience rule with a relative improvement tolerance of 1% (i.e., training stops when the epoch-mean loss does not improve by at least 1% for a fixed number of checks after a minimum number of epochs).

We optimize hyperparameters (number of hidden layers, width, Adam learning rate, λ_{BC}) with a grid search by training the network with a total of 36 different configurations.

Best hyperparameters from grid search. Table 1 summarizes the best configurations selected by grid search for each method and complexity level, together with the final relative ℓ_2 error and the final training loss.

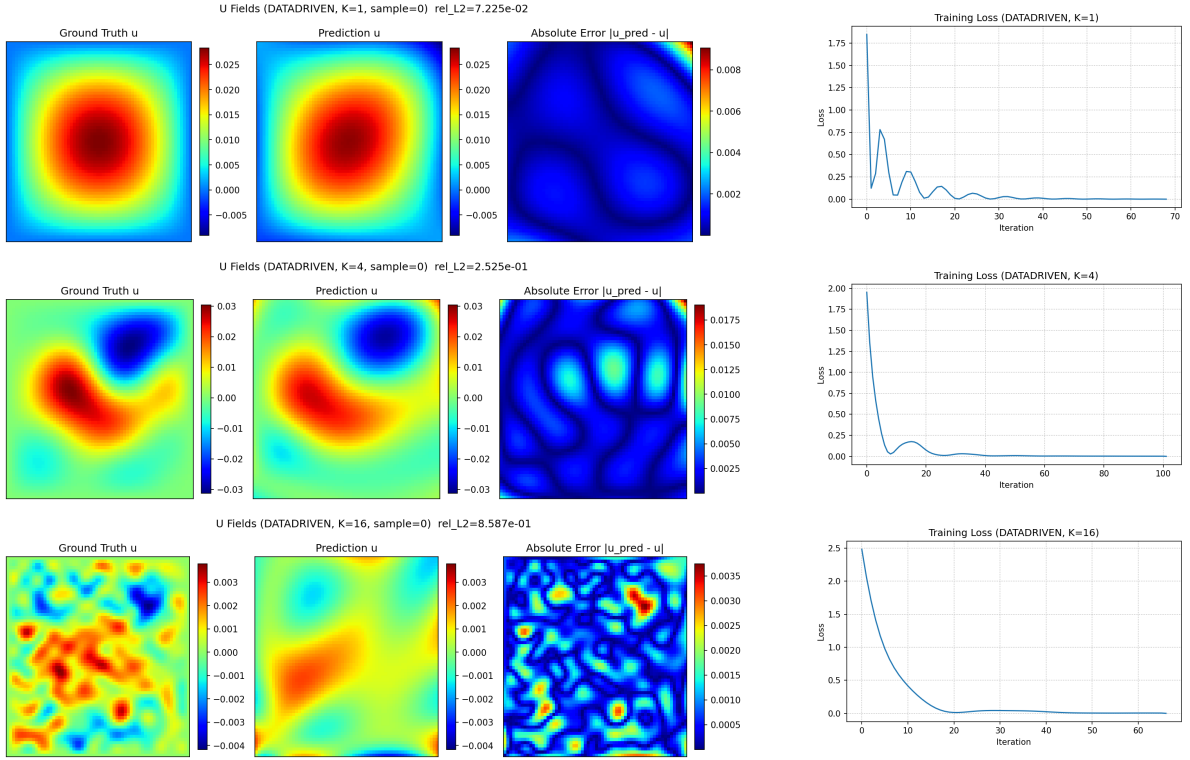
Method	K	Hidden layers	Width	Adam LR	λ_{BC}	Rel. ℓ_2	Final loss
Data-Driven	1	3	128	1e−3	-	0.072	1.01e−6
Data-Driven	4	4	64	5e−4	-	0.252	8.47e−6
Data-Driven	16	4	32	5e−4	-	0.859	1.11e−6
PINN	1	3	32	5e−4	100	0.006	7.09e−5
PINN	4	4	32	5e−4	100	0.183	0.0254
PINN	16	3	32	5e−4	100	1.317	1.93

Table 1: Best hyperparameters selected by grid search for Task 2.

Analysis. As shown in Figure 2, we compare, for each method and each complexity level $K \in \{1, 4, 16\}$, the predicted solution field $\hat{u}_\theta$ (and its absolute error) with the ground truth, together with the corresponding training loss curve.

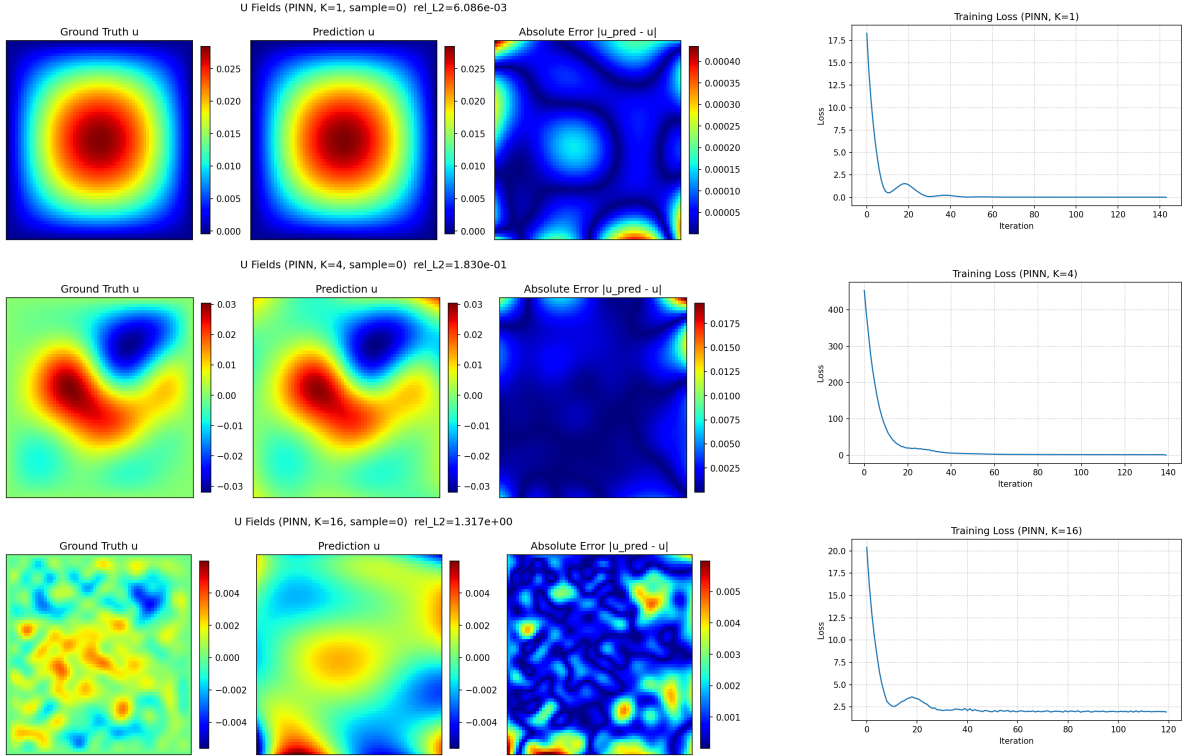
PINNs outperform the data-driven fit for the low and medium regimes in terms of relative ℓ_2 error: for $K = 1$, the PINN reaches 0.006 versus 0.072 for Data-Driven, and for $K = 4$ it achieves 0.183 versus 0.252. In the high-complexity case $K = 16$, both approaches degrade substantially (Data-Driven: 0.86, PINN: 1.32), which is consistent with the fact that the solution exhibits finer-scale spatial variations that are harder for a standard coordinate-based MLP to represent. Moreover, in the PINN setting the PDE loss depends on second derivatives (Laplacian), making optimization more sensitive when high-frequency components are present.

Data-Driven



(a) Data-Driven results for $K \in \{1, 4, 16\}$. Each row shows the field comparison (left) and the training loss curve (right).

PINN



(b) PINN results for $K \in \{1, 4, 16\}$. Each row shows the field comparison (left) and the training loss curve (right).

Figure 2: Results for Data-Driven training (top) and PINN training (bottom).

Task 3 (BONUS) - Loss Landscape Visualization

To visualize the geometry of the loss around the converged (or stuck) parameters θ^* , we follow the loss-landscape methodology of Li et al. (2018) and evaluate the loss on a 2D plane:

$$g(\alpha, \beta) = \mathcal{L}(\theta^* + \alpha \delta + \beta \eta), \quad (\alpha, \beta) \in [-1, 1]^2.$$

where δ and η span two directions in parameter space. We then compute $g(\alpha, \beta)$ on a regular grid and report 2D contour plots of $\log_{10}(g)$.

Rather than using random directions, we use *principal directions* obtained via PCA on the optimization trajectory (snapshots of parameters collected during Adam training). This choice is a consequence of the observation in Li et al. (2018) that random directions can be uninformative in high dimensions, while PCA captures the dominant directions of variation of θ_t around θ^* .

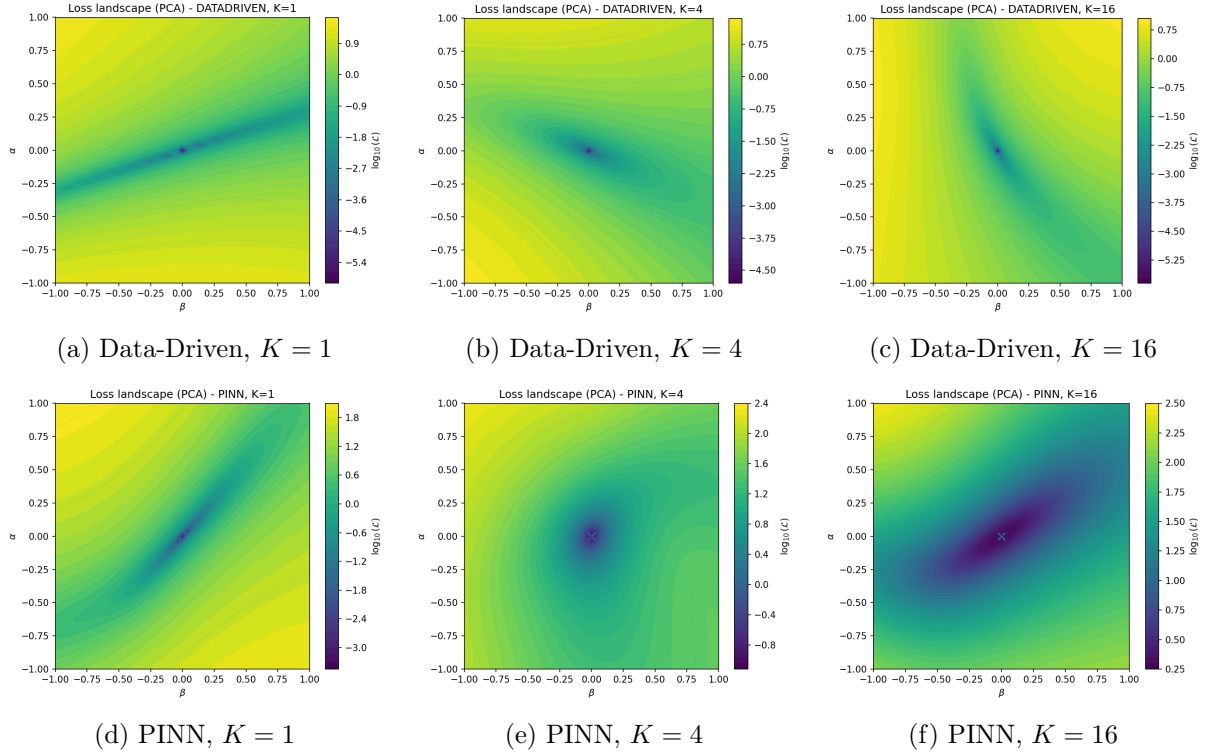


Figure 3: 2D loss landscapes $g(\alpha, \beta)$ (contours of $\log_{10}(\mathcal{L})$) around θ^* for Data-Driven and PINN models across $K \in \{1, 4, 16\}$, using PCA directions from the Adam training trajectory and $(\alpha, \beta) \in [-1, 1]^2$.

Figure 3 shows 2D loss slices around the converged parameters θ using PCA directions. The **PINN** landscapes are more **sharper and anisotropic** than the **Data-Driven** ones, with tighter basins near θ . Indeed, enforcing second-order derivatives can create **ill-conditioned** optimization geometry. As K increases, both methods have their valley structure more deformed, and for $K = 16$ the surfaces shows a **stuck regime**, the loss does not change much near θ^* . This is exactly the failure modes phenomenon reported for PINNs where the physics-based regularization can make training increasingly difficult as the target becomes more complex.

2 - Training an FNO to Approximate a Dynamical System

Task 1 — One2One training: learning $G : u_0 \mapsto u(t = 1)$.

In this task, we train a *1D Fourier Neural Operator* (FNO) to approximate the mapping from an initial condition $u_0(x)$ to the final state $u(x, t = 1)$. The network input has shape $(S, 2)$: the first channel contains the initial condition $u_0(x)$ sampled on a 1D grid, and the second channel contains the spatial coordinates $x \in [0, 1]$. A linear lifting layer `Linear(2→width)` projects these two input channels into a latent representation of dimension `width`. The core of the model consists of four FNO blocks. Each block combines (i) a spectral convolution that performs a real FFT along the spatial dimension, multiplies the lowest `modes` Fourier coefficients by learned complex weights, and returns to physical space via an inverse FFT, and (ii) a pointwise 1×1 convolution (`Conv1d`) in physical space. The two paths are added and passed through a `tanh` nonlinearity. Finally, a pointwise projection head (`Linear(width→64)` followed by `Linear(64→1)`) maps the latent features back to the scalar field $u(x, t = 1)$.

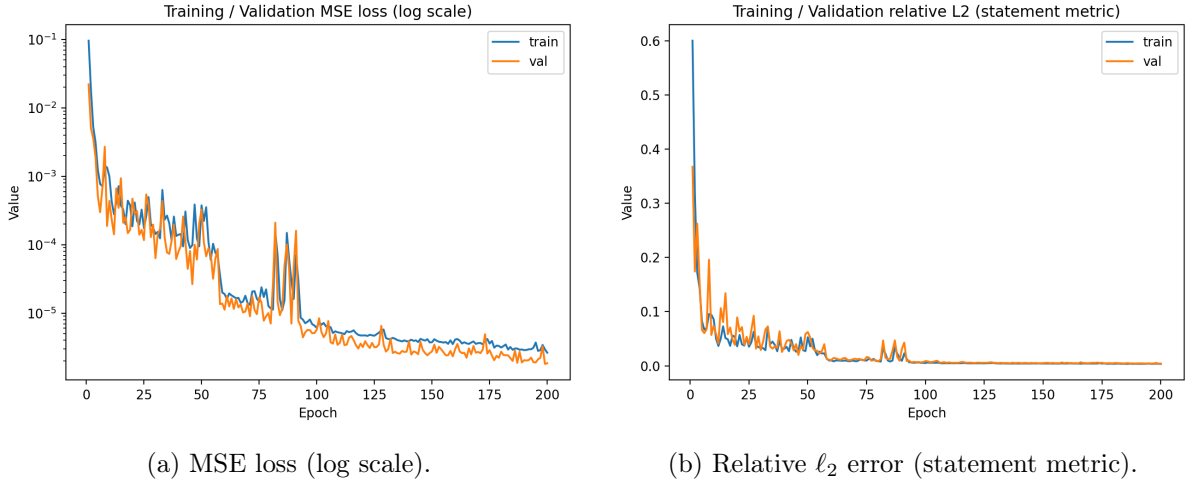


Figure 4: Training dynamics for Task 1.

Figure 4 shows convergence. On the left, the MSE decreases rapidly within the first epochs and then converges at a very low value (around $1e-5$ for both train and validation, thus the model quickly captures the underlying operator). On the right, the relative ℓ_2 error (defined as $\|u_{\text{pred}} - u_{\text{true}}\|_2 / \|u_{\text{true}}\|_2$ and averaged over trajectories) follows the same trend and remains small, suggesting good generalization.

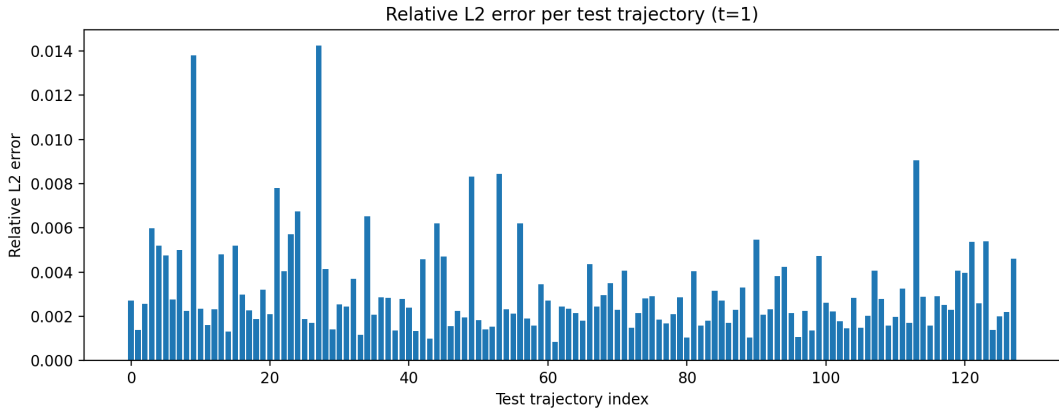
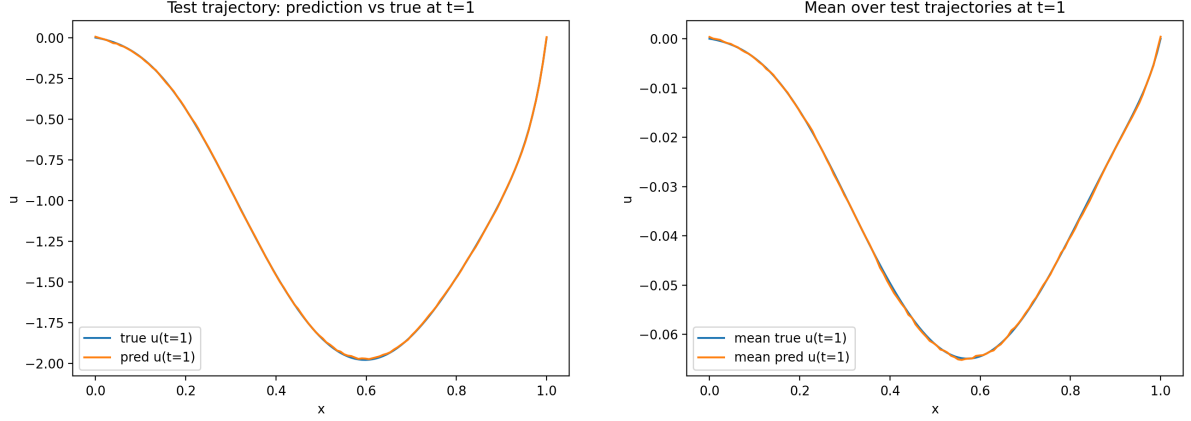


Figure 5: Relative ℓ_2 error per trajectory on the test set (128 trajectories).

Figure 5 reports the relative ℓ_2 error for each of the 128 test trajectories. Most errors lie below 1% (with only two trajectories with larger values around 1,4% which is reasonable). On average, the model achieves about **0.3%** relative error on the test set, which is very low and confirms that the FNO accurately learns the operator $u_0 \mapsto u(t=1)$ on this data distribution. We can visualize this on Figure 6 where the prediction on one test trajectory, and averaged along all trajectories are close to expected ones.



(a) Prediction vs. ground truth on one test trajectory ($t=1$). (b) Prediction vs. ground truth for the mean test trajectory ($t=1$).

Figure 6: Qualitative comparison of FNO predictions at the final time.

Task 2 - Testing on Different Resolutions

We evaluate the FNO model trained in Task 1 (trained at resolution $s=128$) on the test sets `data_test_{s}.npz` for $s \in \{32, 64, 96, 128\}$, and report the average relative ℓ_2 error at the final time $t=1$. As shown in Figure 7, the error increases progressively as the resolution decreases, reaching more than 20% at $s=32$. This behavior is expected because the model was optimized on trajectories discretized on a 128-point grid and therefore learns an operator tied to that discretization. When testing on coarser grids, the input u_0 contains fewer spatial samples and loses fine-scale information. Moreover, the discrete Fourier representation changes with s (different frequency bins and truncation), which can introduce mismatch between the spectral modes seen during training and those available at test time.

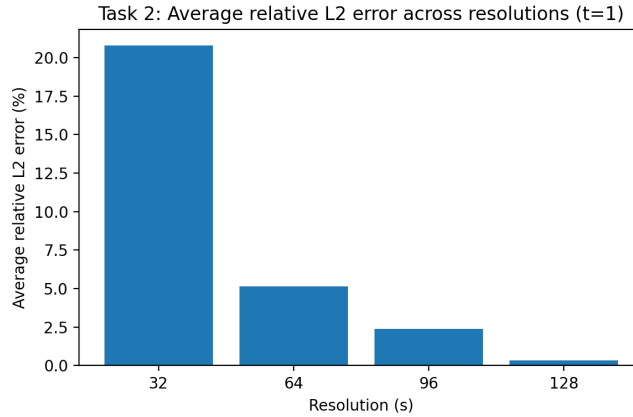


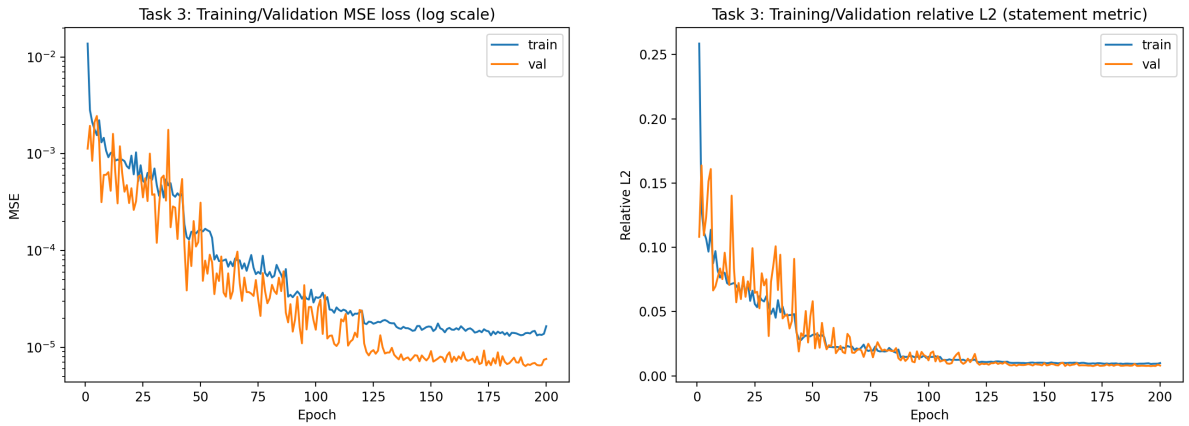
Figure 7: Task 2: average relative ℓ_2 error on the test sets at different spatial resolutions (in %).

Task 3 - All2All Training (Time-Dependent FNO)

In the all-to-all setting, we exploit the flow property of autonomous dynamical systems and train a time-dependent operator to learn transitions between any pair of observed snapshots:

$$u(t_j) \approx S(t_j - t_i, u(t_i)), \quad i < j.$$

For each trajectory, we build training pairs (t_i, t_j) with $i < j$ using the time snapshots $t \in \{0, 0.25, 0.5, 0.75, 1\}$. The model input is a three-channel field of shape $(S, 3)$ containing (i) the state $u(t_i)(x)$, (ii) the spatial grid x , and (iii) the time increment $\Delta t = t_j - t_i$ (broadcast along the spatial dimension). The output is the predicted state $u(t_j)(x)$ (shape $(S, 1)$). To condition the model on time, we use a *Time-Conditional Normalization* layer implemented as a FiLM modulation on top of batch normalization: after normalization, the features are scaled and shifted by time-dependent factors $(\gamma(\Delta t), \beta(\Delta t))$ learned from Δt .



(a) Training/validation MSE loss (log scale).

(b) Training/validation relative ℓ_2 error.

Figure 8: Task 3: convergence of the all-to-all time-dependent FNO.

We observe a clear decrease of both the MSE and the relative ℓ_2 error over training (on training and validation sets), indicating stable convergence.

Test at $t = 1.0$ and comparison with Task 1. We evaluate the all-to-all time-dependent FNO on `data_test_128.npy` by restricting to predictions at the final time $t = 1.0$. Concretely, we feed the model with the initial snapshot $u(t_0)$ and the time increment $\Delta t = 1.0$ (together with the spatial grid), and predict $u(t = 1.0)$. Figure 9 shows the relative ℓ_2 error per trajectory on the test set. On average, the model achieves **about 1%** relative error, which remains correct, but this is more than three times larger than the error obtained in Task 1.

A plausible explanation is that the all-to-all training objective is *harder* than the one2one setting: instead of specializing to the single mapping $u_0 \mapsto u(t = 1)$, the network must learn a family of transition operators $(u(t_i), \Delta t) \mapsto u(t_j)$ for multiple time gaps $\Delta t \in \{0.25, 0.5, 0.75, 1.0\}$. This introduces a broader set of constraints and a potential trade-off in capacity allocation: improving accuracy for intermediate-time transitions may slightly degrade the specific endpoint prediction at $t = 1$.

Finally, we observe that the same trajectories that yielded the largest errors in Task 1 also tend to produce the largest errors under the all-to-all model. In particular, two trajectories still stand out with errors slightly above 4%, while the remaining majority stays close to or below 1%. This suggests that these inputs are more challenging, and that the difficulty persists across training paradigms.

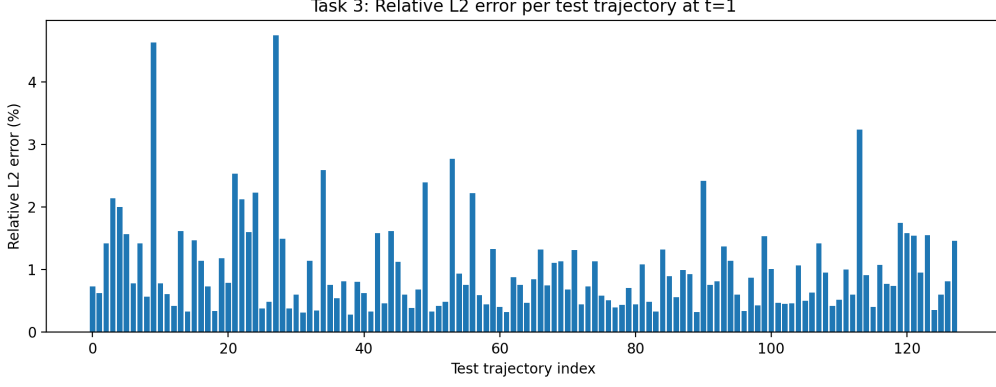


Figure 9: Task 3: relative ℓ_2 error per test trajectory at $t = 1.0$ (in %).

Predictions for several time steps. Figure 10 shows that the average relative ℓ_2 error increases progressively as the prediction time t grows. A natural explanation is that predicting further in time is a harder extrapolation problem: the dynamics has had more time to evolve and small approximation errors in the learned operator translate into larger errors at later times. However, the error remains quite low (between 0.6% and 1%), which indicates that the time-dependent FNO provides accurate forecasts across all considered time horizons.

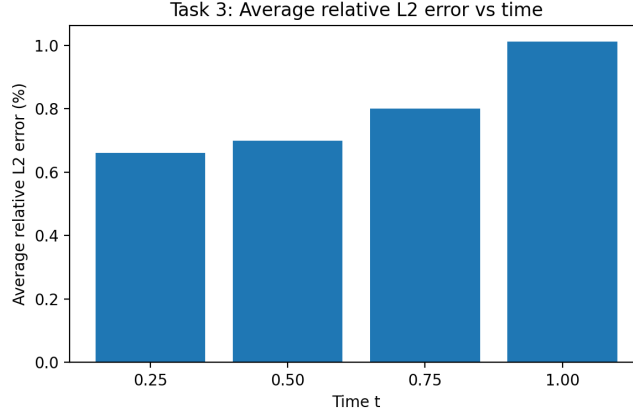
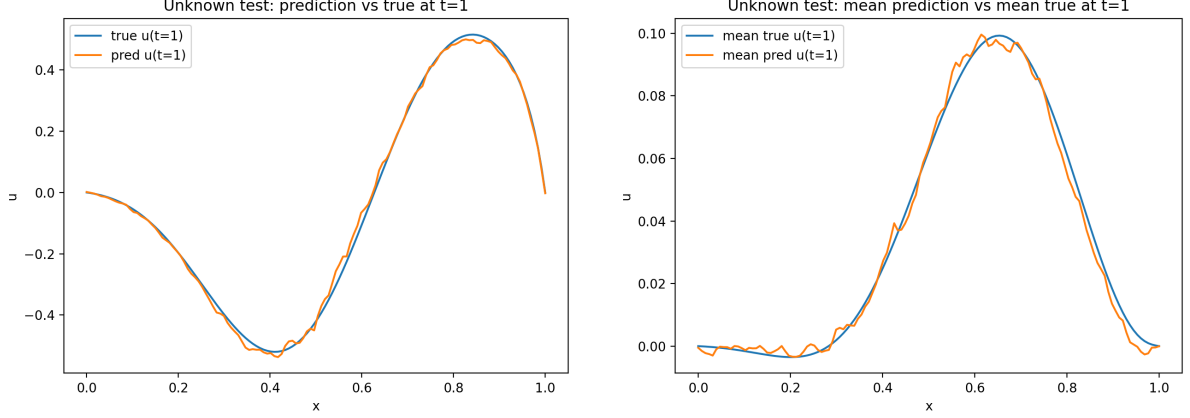


Figure 10: Task 3: average relative ℓ_2 error (in %) on `data_test_128.npy` as a function of the prediction time t .

Task 4 - Finetuning (Unknown Initial Distribution)

Zero-shot evaluation on an unknown initial-condition distribution. We evaluate the all-to-all time-dependent FNO (trained on the original training distribution) in a *zero-shot* manner on the dataset `data_test_unknown_128.npy`. We focus on predictions at the final time $t = 1.0$ by feeding the model with the initial snapshot $u(t_0)$ together with the time increment $\Delta t = 1.0$ (and the spatial grid), and we measure the average relative ℓ_2 error on the 128 test trajectories. We obtain an average error of **6.9%**. This degradation is expected since the model is evaluated on initial conditions drawn from an a priori different distribution than the one used during training, hence the learned operator does not generalize perfectly in a zero-shot setting.

As illustrated in Figure 11, the predicted trajectories remain only approximate and differ from the ground truth, both on an individual example (left) and on the average behavior across the test set (right).



(a) Prediction vs. ground truth for one unknown test trajectory at $t = 1.0$. (b) Prediction vs. ground truth for the mean unknown test trajectory at $t = 1.0$.

Figure 11: Zero-shot qualitative results on the unknown distribution.

Finetuning on the unknown distribution (all-to-all). To adapt the time-dependent FNO trained in Task 3 to the shifted initial-condition distribution, we finetune the pretrained model on the 32 trajectories of `data_finetype_train_unknown_128.npy` using the same all-to-all setup as in Task 3. Concretely, we build training pairs (t_i, t_j) with $i < j$ and train the model to predict $u(t_j)$ from the input $(u(t_i), x, \Delta t = t_j - t_i)$, starting from the Task 3 checkpoint and continuing gradient-based optimization with a smaller learning rate. During finetuning, we freeze BatchNorm running statistics to avoid unstable or biased normalization estimates on the very small finetuning set (this improved generalization as only a few trajectories are available). As shown in Figure 12, the training relative error quickly decreases and becomes much smaller than the validation error. This gap is expected: with only 32 trajectories, the model can fit the finetuning training transitions very closely, while the validation set (8 trajectories) provides a harder and noisier estimate of generalization. Despite this overfitting tendency, finetuning significantly improves performance on the unknown test set: the relative ℓ_2 error at $t = 1.0$ drops to **3.73%**, which corresponds to an almost 50% reduction compared to the zero-shot error (6.9%).

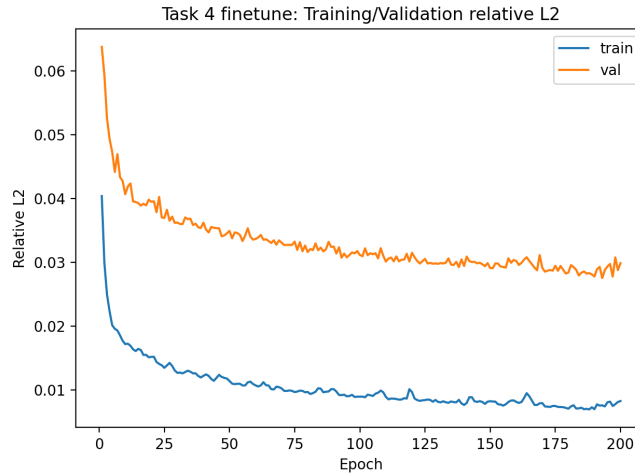


Figure 12: Task 4 (finetuning): training/validation relative ℓ_2 error on the unknown-distribution finetuning data (all-to-all).

Bonus We train a new time-dependent FNO model *from scratch* using only the 32 trajectories from `data_finetune_train_unknown_128.npy`, following the same all-to-all formulation as in Task 3. Each training sample corresponds to a pair of time indices (t_i, t_j) with $i < j$, where the input is the three-channel field $(u(t_i)(x), x, \Delta t = t_j - t_i)$ and the target is the future state $u(t_j)(x)$. We monitor the training and validation relative ℓ_2 error during optimization.

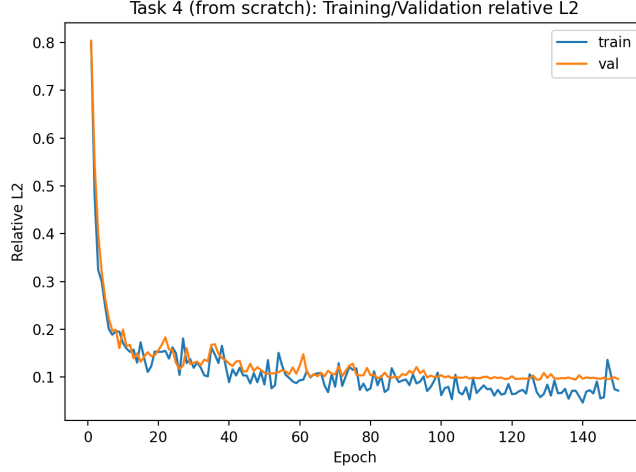


Figure 13: Bonus: training/validation relative ℓ_2 error on the unknown-distribution finetuning data (all-to-all).

We obtain a test error of **15.4%** on `data_test_unknown_128.npy` at $t = 1.0$, which is much higher than the performance achieved with finetuning. This degradation is expected: with only 32 training trajectories, there is not enough data diversity to learn a reliable time-dependent operator from scratch, leading to poor generalization on the larger unknown test set.

Transfer learning is successful because finetuning the pretrained Task 3 model leads to a much lower unknown-test error ($\approx 3.7\%$) than training from scratch on only 32 trajectories ($\approx 15.4\%$).

3 - Geometry-Aware Operator Transformer (GAOT)

Task 1 - Official GAOT Baseline (Strategy I)

We train the official GAOT implementation (by using the <https://github.com/camlab-ethz/GAOT> codebase) to have a baseline on the Elasticity dataset, keeping the default hyperparameters from the repository and running the training for 100 epochs.

Model (Strategy I)	Test Relative L1 Error	# Tokens	Training time
GAOT (default)	0.0624	4096 (latent) / 1024 (ViT)	33m05s

Table 2: Baseline performance on the Elasticity dataset .

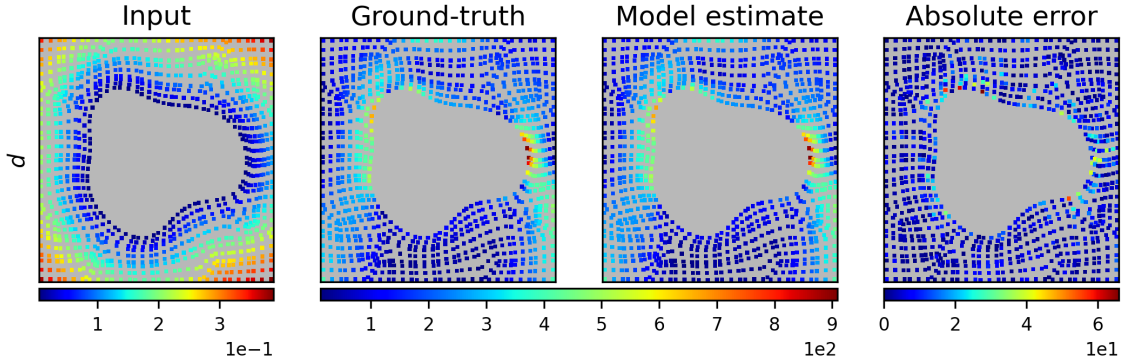


Figure 14: Results on Elasticity.

We can see on Fig. 14 that the predicted field (*Model Estimate*) matches the *Ground Truth* closely, with only minor errors.

Task 2 - Random Sampling Tokenization & Dynamic Radius (Strategy II)

(a) Random sampling tokenization

We implemented *Strategy II* where the latent tokens are no longer defined on a fixed meshgrid. Instead, for each sample we randomly select N latent coordinates $\{y_\ell\}_{\ell=1}^N \subset \mathcal{D}$ (256 in our example). This enables tokenization on irregular geometries without relying on a regular $H \times W$ layout. These tokens serve as the latent support: MAGNO encodes physical nodes to the sampled latent tokens, the Transformer processes the latent representation, and MAGNO decodes back to the physical/query coordinates.

Code modifications in GAOT repository.

- `src/datasets/data_processor.py`: Added random latent token generation and propagated latent tokens through the dataloading pipeline.
- `src/datasets/data_utils.py`: Extended the variable-coordinate dataset and collate function to carry sampled latent tokens in each batch. The `vx` batch now includes an additional `latent_batch` tensor of shape `[B, N, 2]`.
- `src/datasets/graph_builder.py`: Adapted graph construction to support non-grid latent queries.

- `src/trainer/static_trainer.py`: Updated train/validation/test steps for vx mode to unpack the extra `latent_batch` and pass it to the model as `latent_tokens_coord`. Enabled the use of precomputed graphs (through `precompute_edges`) to avoid rebuilding neighbors at runtime for batchwise latent tokens.
- `src/model/layers/magno.py`: Modified MAGNO encoder/decoder to accept batchwise latent token coordinates in vx mode (either `[N,2]` shared or `[B,N,2]` per batch). The vx path now uses per-sample latent coordinates when computing message passing and geometric embeddings.

Practical note (ViT processor constraint). The current GAOT processor follows a ViT-style reshape and thus expects $N = H \cdot W$. For random tokens we therefore set `latent_tokens_size` to match the chosen N (16×16 for $N = 256$) and used `patch_size=1` to prevent additional token downsampling. This preserves compatibility with the official architecture while enabling Strategy II tokenization.

(b) Dynamic radius

With random latent tokens, using a fixed radius r to build the encoder/decoder graphs can lead to *holes* when r is too small, or *large overlap* when r is too large. To address this, we implement a **Dynamic Radius** strategy inspired by RIGNO: for each token y_ℓ , we compute a local radius

$$r_\ell = \alpha \cdot d_k(y_\ell), \quad (1)$$

where $d_k(y_\ell)$ is the distance from y_ℓ to its k -th nearest neighbor (KNN distance), and α is a global scaling factor. Sparse regions get larger radii to maintain connectivity, while dense regions keep smaller radii to preserve locality.

Code modifications in GAOT repository. We implemented this strategy in the variable-coordinate (VX) pipeline:

- `src/datasets/graph_builder.py`: Added KNN-based radius computation (d_k) and used per-token radii to build encoder and decoder neighbor lists.
- `src/model/layers/utils/neighbor_search.py`: Ensured the neighbor search supports *per-query* radii vectors (i.e., a different radius for each query point), by routing tensor radii to implementations that can handle it.
- `src/trainer/static_trainer.py`: Forwarded the dynamic-radius configuration to the graph builder so that graphs are built with the requested strategy.
- Configuration (`elasticity_strategy_2.json` + config dataclasses): added the dynamic-radius hyperparameters (`dynamic_radius_k`, `dynamic_radius_alpha`).

Connectivity analysis (coverage and degree). To verify that the graph connectivity covers the entire physical domain despite random token placement, we report:

- **Coverage**: fraction of physical nodes that are connected by at least one encoder edge (i.e., appear in the union of neighbors of the latent tokens).
- **Degree**: number of physical neighbors per latent token in the encoder graph (min/mean/max across tokens).

Coverage is consistently $\approx 98\%$ – 99.7% while keeping a moderate mean degree (≈ 11). This shows that the adaptive radii successfully avoid disconnected regions without creating excessive overlap.

Table 3: Dynamic-radius statistics and connectivity diagnostics.

Split	#samples	r_{latent} (min/mean/max)	Coverage	Degree (min/mean/max)
Test	256	0.05/0.11/0.27	956/972 = 0.9835	2/10.7/32
Train	1024	0.05/0.11/0.22	969/972 = 0.9969	3/11.9/32
Val	128	0.04/0.11/0.24	957/972 = 0.9846	2/11.3/32

Dynamic radius visualization. Figure 15 shows the learned local radii on one sample (coordinates in the scaled domain $[-1, 1]^2$). **Latent tokens** correspond to the randomly sampled operator tokens used as graph queries for the encoder (physical $\rightarrow$ latent). **Physical queries** correspond to the original mesh points used as graph queries for the decoder (latent $\rightarrow$ physical). For readability, the physical plot is shown with a random subset of 300 points. We can see that radii increase in sparser regions.

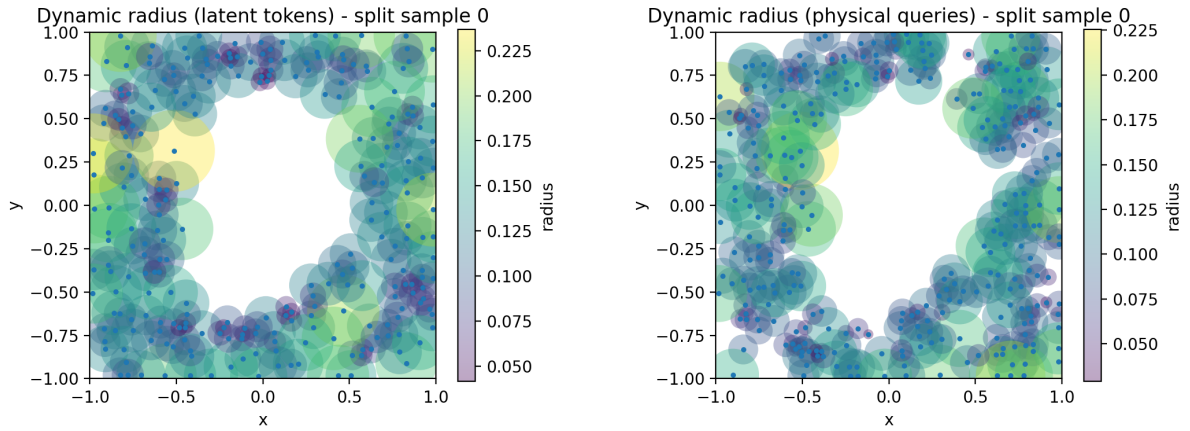


Figure 15: Dynamic-radius fields. **Left:** radii associated with randomly sampled latent tokens (encoder queries). **Right:** radii associated with physical points (decoder queries), displayed with a 300-point subset for visibility.

Table 4: Final training setup and results (Random Sampling + Dynamic Radius).

Model (Strategy II)	Test Relative L1 Error	#Tokens	Training time
GAOT (RS + Dynamic Radius)	1.21	256	26m04s

We then train the model for 100 epochs using **Random Sampling tokenization** (*without* resampling at each epoch) together with the **Dynamic Radius** strategy to build the encoder/decoder graphs.

Compared to the first test, results are slightly worse (by a factor 2, which can be explained by the fact that random tokenization introduces *sampling variance* (some regions may be under-represented by the selected tokens), and dynamic radii can lead to *non-uniform local connectivity* (varying neighborhood sizes). Both of which make optimization noisier than a regular grid-based latent layout. Figure 16 reports the comparison between ground-truth and model prediction on an example.

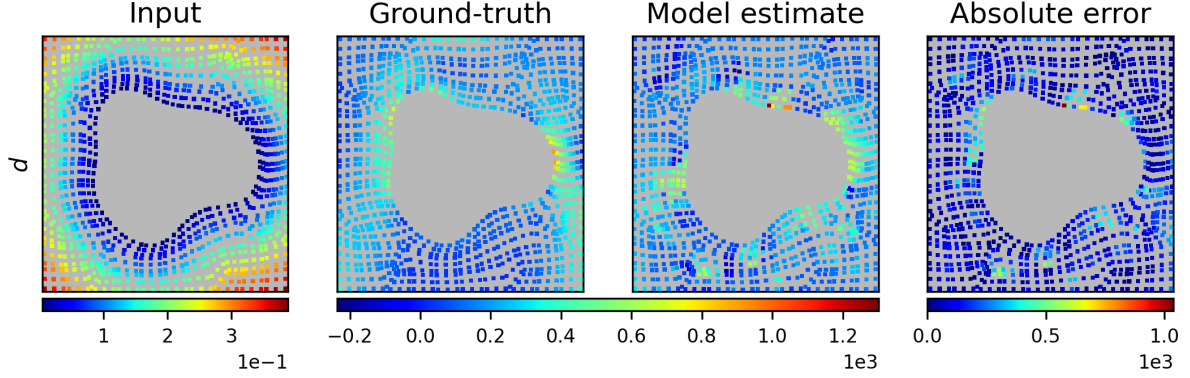


Figure 16: Elasticity example: input field, ground-truth, model prediction, and absolute error.

Task 3 (BONUS) - Positional Encoding and Sequence Length Reduction

In Strategy II, the latent tokens are *randomly sampled* in the continuous physical domain, so regular grid indices are no longer available. To inject geometry, we evaluate (i) an **Absolute Positional Encoding** obtained from a coordinate MLP, (ii) **continuous relative position** mechanisms using RoPE or a Euclidean-distance relative bias, and (iii) a **Cross-Attention** compression layer to reduce the effective sequence length when the number of random tokens becomes large.

Tested configurations. All experiments are run with Strategy II (random tokens) and the Dynamic Radius graph construction. We report the *test relative error* (same metric as previous tasks), the number of latent tokens, and the total training time.

Model / Positional Encoding	Test Relative Error	# Tokens	Training Time
Absolute PE (Coord-MLP)	0.24	256	26m42s
RoPE (continuous coords), no rel. bias	0.39	256	26m57s
RoPE + Euclidean relative bias	0.33	256	26m56s
Coord-MLP + Cross-Attention (64 seeds)	0.60	256 $\rightarrow$ 64	26m35s

Table 5: Bonus ablation study on positional encoding and attention design for Strategy II.

Absolute PE via coordinate MLP. We implement an MLP that maps each coordinate vector $x \in \mathbb{R}^2$ to an embedding in the hidden dimension D , and add it to the token features prior to the Transformer blocks. This approach led to the best performance among the PE-only variants, with a test relative error of 0.24.

Continuous relative position: RoPE and distance bias. We apply RoPE in the attention module while using the continuous token ordering (random tokens) and investigate whether the resulting inductive bias remains beneficial. In our setting, RoPE alone underperforms the coordinate-MLP absolute PE (0.39 vs. 0.24). We then add a **relative bias term** computed from pairwise Euclidean distances $\|x_i - x_j\|$ and injected in the attention logits. This improves RoPE, reducing the test error to 0.33.

Cross-Attention compression (Perceiver-style). When patching is disabled, the sequence length scales with the number of random tokens. To mitigate this, we implement a Cross-Attention layer that maps the N token embeddings to a smaller set of M *seed* tokens ($M \ll N$),

which are then processed by the Transformer. Seed tokens attend to all token embeddings (keys/values) to produce a compact representation, which reduces the effective attention complexity inside the Transformer from $\mathcal{O}(N^2)$ to $\mathcal{O}(M^2)$ (plus the cross-attention $\mathcal{O}(MN)$). With $N = 256$ tokens and $M = 64$ seeds, this variant reaches a test error of 0.60.

We also visualize the prediction for each configuration :

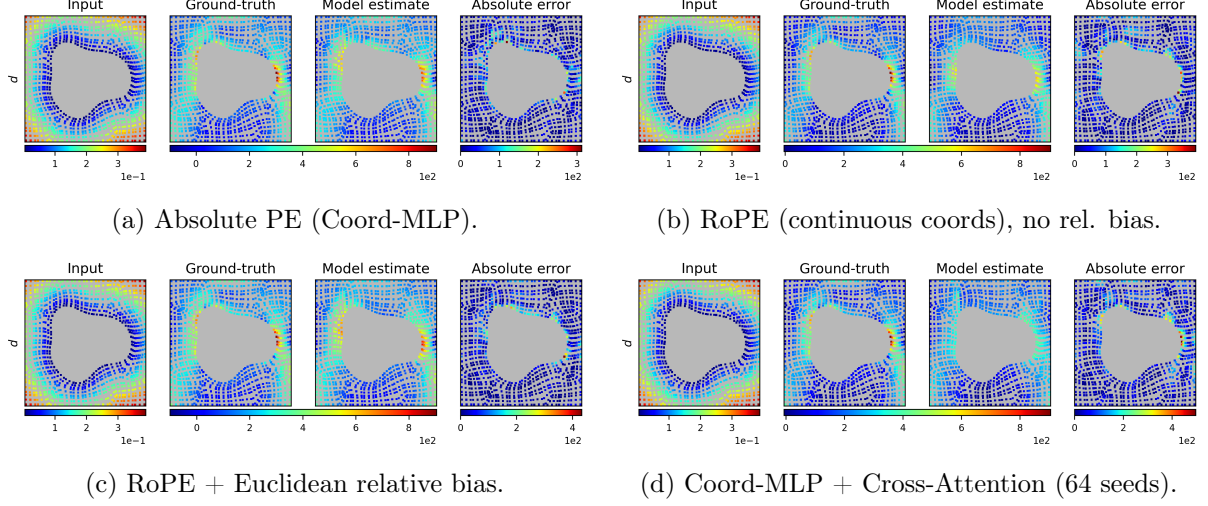


Figure 17: Qualitative comparison of the bonus variants on Elasticity (Strategy II).

The **coordinate-MLP absolute PE** performs best in this setting. A plausible explanation is that it provides a *continuous and smooth* mapping from geometry to the hidden space, which makes sense with irregular token layouts. RoPE, when applied without discrete index structure, appears less suited to this irregular sampling. Finally, **Cross-Attention** reduces the effective sequence length by compressing 256 tokens into 64 seed tokens, but this compression can introduce an information bottleneck. This likely explains the higher error in our experiment.

References

- [1] Krishnapriyan et al. (2021). *Characterizing possible failure modes in physics-informed neural networks*. NeurIPS.
- [2] Li et al. (2018). *Visualizing the loss landscape of neural nets*. NeurIPS.
- [3] Codebase: <https://github.com/camlab-ethz/GAOT>
- [4] Wen et al. (2025). *Geometry Aware Operator Transformer as an efficient and accurate neural surrogate for PDEs on arbitrary domains*. arXiv:2505.18781.
- [5] Mousavi et al. (2025). *RIGNO: A Graph-based framework for robust and accurate operator learning for PDEs on arbitrary domains*. arXiv:2501.19205.
- [6] Qi et al. (2025). *SpiderSolver: A Geometry-Aware Transformer for Solving PDEs on Complex Geometries*. NeurIPS.